Danmarks Tekniske Universitet

DTU

02443 Stochastic Simulation

Report - Project 2, CR: Bo Friis Nielsen & Nicolai Siim Larsen, DTU 2026

Alexander Husted - s253831
August Pallesen - s252947
Kristian Juul Rasmussen - s204638
Mona Saleem Aljsine - s204226

October 22, 2026

All members contributed equally to this work.



Figure 1: Hakkar the Soulflayer in the dungeon Zul'Gurub in the video game, World of Warcraft.

Introduction

In this project, we aim to model the *Corrupted Blood incident*, a virtual epidemic that occurred in the multiplayer online game *World of Warcraft* (WoW) in 2005.

A common gameplay structure for players in WoW is to collaborate with other players in dungeons or raids. These are shared game areas where a group of players works together to defeat enemies and complete challenges. Towards the end of such an encounter, there is usually a particularly strong enemy, referred to as a boss, which often requires the entire group of players to work together.

However, in 2005, World of Warcraft experienced a software glitch that is still studied in the field of epidemic modeling. The boss *Hakkar the Soulflayer*, located in the raid *Zul'Gurub*, had an ability called *Corrupted Blood*, which was designed specifically for the fight and intended to pose a special challenge for the players. The effect damaged infected players over time and could spread to nearby allies. What this effectively meant, in simple terms, was that the disease could spread rapidly between players if they stood too close to each other. This forced players to distance themselves and play tactically during the encounter.

Normally, harmful effects from bosses are either removed after the encounter or remain confined to the dungeon or raid. This is where the glitch came in. One player class, the hunter, was able to have a beast companion, known as a pet. In addition, warlock minions could also be affected. These companions could become infected with *Corrupted Blood*, and under certain circumstances the affliction was not removed when leaving the raid. As a result, the debuff could be brought outside of *Zul'Gurub* and into the wider World of Warcraft world, where it started to spread among players and non-player characters. In this way, a mechanic that was intended to remain inside a single boss fight began to mimic the dynamics of a real epidemic.

The *Corrupted Blood incident* is still studied because it differs from many artificial disease simulations by including human behaviour and reactionary elements. As the disease spread, interesting patterns began to emerge. Some players avoided densely populated areas, some spread information about high-risk zones, and others gathered in designated areas where healing could occur or where they believed it was safe. The incident became particularly interesting to epidemiologists because players reacted differently depending on factors such as level, equipment, experience, and individual behaviour.

For this reason, the *Corrupted Blood incident* provides a useful motivation for studying epidemic models through stochastic simulation. In this project, we use the incident as inspiration for building compartment-based disease models, where players can move between states such as susceptible, infected, recovered, or dead. We begin with simpler classical epidemic models and then consider possible extensions that better reflect the virtual epidemic setting, such as additional disease states, different contagion rates, and behavioural differences between individuals. The aim is to investigate how random interactions, parameter choices, and model assumptions influence whether a disease spreads, disappears, or becomes severe enough to affect a large part of the population.

0.1 The basic model : From SIR to SIRD to SISD

A classical SIR model divides the population into three compartments namely, susceptible individuals S, infected individuals I, and recovered individuals R. A deterministic SIR model can be written as following

$$\frac{dS}{dt} = -\frac{\beta}{N}SI$$

$$\frac{dI}{dt} = \frac{\beta}{N}SI - \gamma I$$

$$\frac{dR}{dt} = \gamma I$$

Where, β is the infection rate between infected and susceptible individuals, γ is the recovery rate, and N is the total population size. In the classical SIR model, recovered individuals move to the recovered compartment R, meaning that they are no longer susceptible to infection.

Before constructing the stochastic simulation model, it is useful to consider a discrete-time version of the deterministic model. In this formulation, the continuous rates of change are replaced by step-by-step recursive updates from time step t to $t+1$. Using a time step of length Δt, the model becomes

$$\begin{aligned} S_{t+1} &= S_t - \Delta t \frac{\beta}{N} S_t I_t \\ I_{t+1} &= I_t + \Delta t \left(\frac{\beta}{N} S_t I_t - \gamma I_t \right) \\ R_{t+1} &= R_t + \Delta t \gamma I_t \end{aligned}$$

As we present new additions to the model, we will use the deterministic equations to motivate the possible transitions and rates in the stochastic simulation.

To account for death explicitly, the SIR model can be extended to a SIRD model. In this model, infected individuals can either recover or die. The deterministic SIRD model is given as follows

$$\begin{aligned} \frac{dS}{dt} &= -\frac{\beta I S}{N}, \\ \frac{dI}{dt} &= \frac{\beta I S}{N} - \gamma I - \mu I, \\ \frac{dR}{dt} &= \gamma I, \\ \frac{dD}{dt} &= \mu I. \end{aligned}$$

Where μ is the death rate for infected individuals. Thus the discrete model can be expressed as follows

$$\begin{aligned} S_{t+1} &= S_t - \Delta t \frac{\beta I_t S_t}{N}, \\ I_{t+1} &= I_t + \Delta t \left(\frac{\beta I_t S_t}{N} - \gamma I_t - \mu I_t \right), \\ R_{t+1} &= R_t + \Delta t \gamma I_t, \\ D_{t+1} &= D_t + \Delta t \mu I_t. \end{aligned}$$

However, in the context of World of Warcraft, it is not natural to assume that players become permanently immune after recovering from the disease. Instead, a recovered player should become susceptible again and may be infected later to which we define $N_{alive} = S + I$. Therefore, we modify the model by replacing the transition $I \to R$ with the transition $I \to S$. This gives an SISD model, where players are either susceptible, infected, or dead expressed as follows

$$\begin{aligned} \frac{dS}{dt} &= -\frac{\beta I S}{N_{alive}} + \gamma I, \\ \frac{dI}{dt} &= \frac{\beta I S}{N_{alive}} - \gamma I - \mu I, \\ \frac{dD}{dt} &= \mu I. \end{aligned}$$

Thus the model can be expressed as follows

$$S_{t+1} = S_t + \Delta t \left(-\frac{\beta I_t S_t}{N_{alive}} + \gamma I_t \right),$$

$$I_{t+1} = I_t + \Delta t \left(\frac{\beta I_t S_t}{N_{alive}} - \gamma I_t - \mu I_t \right),$$

$$D_{t+1} = D_t + \Delta t \mu I_t.$$

In the following, we will define our assumptions and parameter estimates for the SISD model.

0.2 Simulation approach for baseline model

We model the disease using a continuous-time stochastic simulation approach, similar to the CTMC approach used in Project 1. However, a few modifications are needed to make the method fit the SISD epidemic model. In this model, the possible transitions are infection, recovery by going back to S, and death:

$$S \to I, \qquad I \to S, \qquad I \to D$$

At each step of the simulation, the current state is given by (S, I, D). Since dead players can no longer participate in the epidemic, we define the living population as described earlier $N_{alive} = S + I$. The simulation is performed as follows:

1. We set the while loop to be true as long as the number of infected players is larger than zero, meaning $I > 0$, and as long as the maximum number of minutes has not been reached. If $I = 0$, the disease has disappeared from the population.

2. We set the rates based on the SISD differential equations. The infection rate, recovery rate, and death rate are given by $I_r = \frac{\beta SI}{N_{\text{alive}}}, \quad S_r = \gamma I, \quad D_r = \mu I$. We then sum the rates to obtain the total rate $T_r = I_r + S_r + D_r$.

3. We use the total rate T_r to pick the sojourn time. The time until the next event is sampled as $\tau \sim \text{Exp}(T_r)$, where the expected waiting time is $E[\tau] = \frac{1}{T_r}$.

4. Now we convert the rates into probabilities of continuing to a given state, with states (S, I, D). And draw a random variable $u \sim Uniform(0, 1)$ to pick the state. Based on the state, we adjust (S,I,D) accordingly.

$$\begin{cases} (S, I, D) \to (S - 1, I + 1, D), & \text{if } u < \dfrac{I_r}{T_r}, \\ (S, I, D) \to (S + 1, I - 1, D), & \text{if } \dfrac{I_r}{T_r} \le u < \dfrac{I_r + S_r}{T_r}, \\ (S, I, D) \to (S, I - 1, D + 1), & \text{if } \dfrac{I_r + S_r}{T_r} \le u < 1. \end{cases}$$

Part 0 - initial model framework and initial simulations

0.3 Assumptions

The following describes the assumptions made for the duration of infection time along with the interpretation of death in the basic modeling for Part 1 in section 1.

0.3.1 Duration of infection

In WoW, the *Corrupted Blood* debuff infected a player for 4 seconds. However, a player could be re-infected instantly after the expiration of the debuff, if they remained close to infected players or NPCs. Instead of modeling this dynamic, we will simply assume that a person can have the disease for a random amount of time with an expected time given as follows [Wikb].

$$T_i \sim \text{Exp}(\mu + \gamma), \qquad E[T_i] = \frac{1}{\mu + \gamma}$$

Where, γ is the recovery rate and μ is the death/removal rate. Thus, $\mu + \gamma$ is the total rate at which an infected player leaves the infected state.

0.3.2 Death

In WoW, death is not permanent, since a player can simply respawn after dying. However, players who do not have enough HP (virtual health) to leave an infected area, tend to get stuck in a death loop due to the high damage inflicted from Corrupted Blood. Therefore, we are going to model death as the probability that a player gets stuck into such a death loop and rage-quit/log-out instead of simulating the respawn. In other words, realistically, some players would go from $S \to I \to D \to S$, but we will model this as $S \to I \to D$.

0.4 Setting parameters

The time unit of the model is minutes, the infected state does not represent the exact duration of the debuff, but rather the period in which a player contributes to the outbreak. In the following we are going to roughly estimate some rates based on game information. Since we use minutes as interval, the rates are the minutely effect, meaning that $\beta = 0.8$ means that one person effectively transmits the debuff to 0.8 others per minute.

We found the following parameters

$$\beta = 0.8$$

The infection rate in the game is really high, with 100% probability of infection the other players. However we will need to adjust for the probability that two uninfected people meet. Since this is still high because most people are located near shops and re-spawn areas we scale it to a spreading rate of 0.8. [Wika]

$$\gamma = 0.3$$

The recovery rate in World of war craft are based on the amount of HP a player has. The infection deals 1125 damage instantly and 337 damage every 2 seconds for 4 seconds, 1.799 total. [Wowb] This corresponds roughly to the hp of a level 70 player [Wowa], and if we just assume a uniform distribution of levels. we get minutely recovery rate of $\gamma = 0.3$.

$$\mu = 0.15$$

According to players on Reddit, many people actually didn't log-out, but instead they tried to flee to other areas. [red] So we are going to model it st. approximately $\frac{1}{3}$ of the players leaving the active population rather than recovering. $P(\textit{Leave}) = \frac{\mu}{\gamma+\mu} = \frac{1}{3} \Rightarrow \mu = 0.15$

$$I_0 = 20$$

The outbreaks comes from Zul'Gurub raid which is a 20-player rate. So the initial infection was 20 people.
Simulating this for $N = 100,000$ players once, we achieve following results as seen on Figure 2
From this we get the following trajectories:
Simulating this for $N = 100,000$ over 100 simulations, we achieve the results as seen on Figure 3:

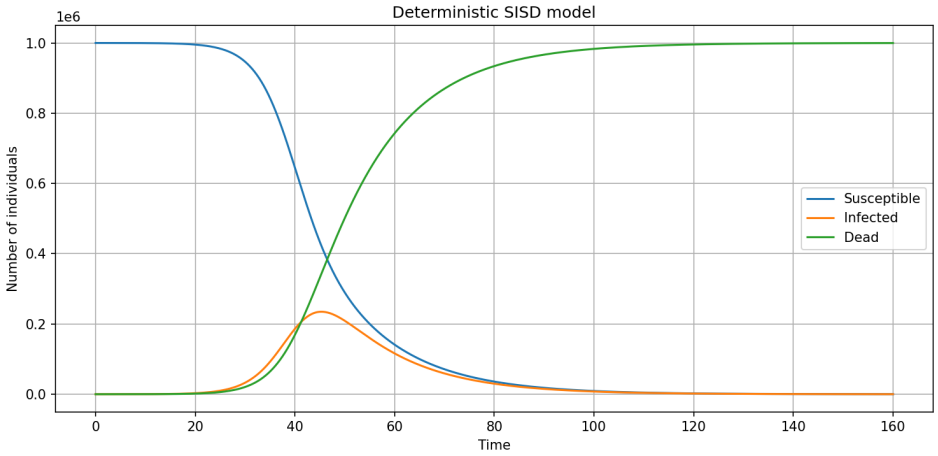
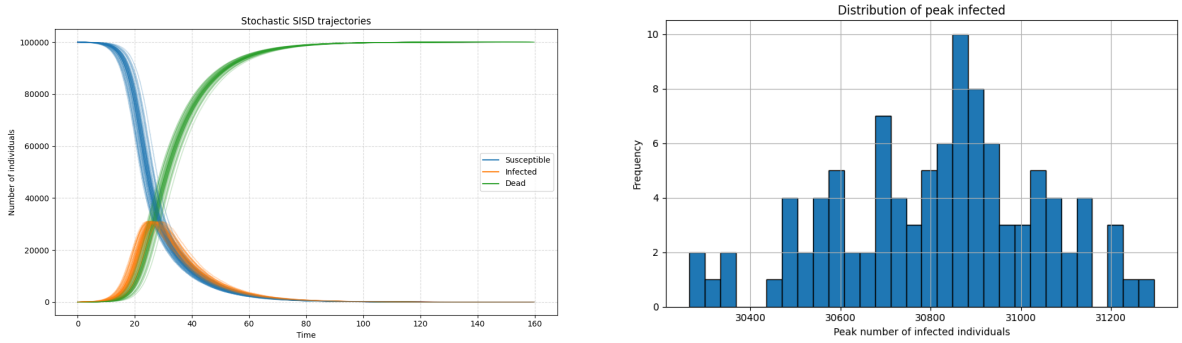


Figure 2: Deterministic SISD model for one simulation of $N = 100,000$ players.



(a) Trajectories

(b) Infections histogram

Figure 3: Results from simulation with N = 100.000 players over 100 simulations

Quantity	**Value**
Average final deaths	99994.33
Average final death fraction	0.9999
Average peak infected	30815.48
Average epidemic duration	152.21

Table 1: Summary statistics over 100 simulations

And we achieve the average statistics as listed in Table 1
Under normal circumstances, an epidemiological model that results in the entire population dead, is often regarded as being potentially faulty do to its consistent extreme outcomes. However, we assure the reader that this degree of fatality is depicting the system as it was, as seen in figure below (a screenshot of the incident).



Figure 4: Player screenshot of the corrupted blood incident

1 Part 1 - basic modeling:

1.1 a: What is the likelihood that the disease will disappear

Under conventional conditions, one can predict whether or not an epidemic will occur via the basic reproduction number R_0. For the conventional SIR model structure, the reproduction number is defined as;

$$R_0 = \frac{\beta}{\gamma}$$

where $R_0 < 1$ no epidemic will occur, and inversely if $R_0 > 1$ and epidemic will occur. What this means in laymans terms is: Whether or not a disease can effectively spread vastly depends on how fast if infects new individuals and how fast it dies out in individuals. If it dies out faster than it can spread, the disease will not be an epidemic. However, if it can survive in the host body long enough to effectively spread to multiple people, an epidemic will occur. That is a sufficient answer for conventional SIR models, but our model different in multiple areas. Its state structure is different, and we also have stochastic elements implemented. As such this specific equation is not sufficient to answer the question. For that reason we instead run a simulation a sufficient number of times, which yields the results seen in table 1. To support this claim we ran 500 simulation of the SISD model, and found that there were no instances where a breakout did not occur.

1.2 b: Can the disease exhibit cyclical behavior

The baseline SISD model does not exhibit sustained cyclical behavior. Although the model allows infected players to recover and become susceptible again through the transition $I \to S$, the death state D acts as an absorbing state. This means that once a player enters the death or removal state, they no longer participate in the epidemic. As a result, the active population gradually decreases over time, and the system will eventually reach a state where the disease disappears or almost all players have been removed from the active population.
In classical epidemic models, cyclical behavior usually requires some mechanism that replenishes the susceptible population or returns individuals from a removed state back to susceptibility. For example,

in an SIRS model, recovered individuals may lose immunity over time and become susceptible again. This can be written as

$$\frac{dS}{dt} = -\frac{\beta IS}{N} + \eta R,$$
$$\frac{dI}{dt} = \frac{\beta IS}{N} - \gamma I,$$
$$\frac{dR}{dt} = \gamma I - \eta R.$$

Here, η is the rate at which recovered individuals lose immunity and return to the susceptible state. Such a feedback mechanism can allow the disease to return after a period of decline.
In our baseline SISD model, there is already a transition from infected back to susceptible, $I \to S$, which means that individuals can be reinfected. However, this alone is not enough to create sustained cycles, because the absorbing death state continuously removes individuals from the active population. Therefore, the model may show temporary fluctuations in the number of infected players, especially in the stochastic simulations, but it does not produce regular cyclical waves.
To make cyclical behavior possible in the World of Warcraft setting, we can extend the model by allowing players in the death state to return to the susceptible state. This can be interpreted as resurrection, since many healers and characters in World of Warcraft have abilities that bring dead players back to life. In this case, the model becomes an SIDS model, with the additional transition $D \to S$. Let η denote the resurrection rate. Then a deterministic version of the model can be written as

$$\frac{dS}{dt} = -\frac{\beta IS}{N_{alive}} + \gamma I + \eta D$$
$$\frac{dI}{dt} = \frac{\beta IS}{N_{\text{alive}}} - \gamma I - \mu I$$
$$\frac{dD}{dt} = \mu I - \eta D,$$

where $N_{\text{alive}} = S + I$. The corresponding discrete-time formulation is

$$S_{t+1} = S_t + \Delta t \left(-\frac{\beta I_t S_t}{N_{alive}} + \gamma I_t + \eta D_t \right)$$
$$I_{t+1} = I_t + \Delta t \left(\frac{\beta I_t S_t}{N_{alive}} - \gamma I_t - \mu I_t \right)$$
$$D_{t+1} = D_t + \Delta t \left(\mu I_t - \eta D_t \right)$$

where $N_{\text{alive},t} = S_t + I_t$. With this extension, the model has a feedback mechanism from D back to S, meaning that the active population can be replenished. This makes cyclical or quasi-cyclical behavior more plausible, since new susceptible players can re-enter the system and become infected again. In the context of the Corrupted Blood incident, this represents the idea that players who died could respawn or be resurrected and then become part of the outbreak again. Lastly, another way cyclical or wave-like behavior could arise is if new players or groups of players enter the system at different times. For example, if batches of players return from dungeons or raids periodically, the susceptible population would be restored over time. This could create repeated waves of infection, even if the baseline SISD model itself does not produce sustained cycles.

1.3 c: For a highly deadly disease can the population be extinguished?

In the context of model conventional SIR model construction this question can be interpreted in multiple ways, but all taks are the same. Change the parameters β (transmission rate) and/or γ (death rate) to make the disease behave considerably more deadly. However, for this question "deadly" can mean multiple things. For a disease to be deadly, it has to have a high likelihood to cause death. But our general understanding of "deadlier" can be multifaceted, because the general understanding is "increased likelihood of causing death", it is a state a disease can reach in multiple

ways. For example, a mutation of an average disease can be considered deadly if it can spread fast and kill millions in the span of a month, even if the death rate is very low.
this increase can present itself in multiple ways:
1) the death rate could remain unchanged but it transmission rate increased
2) the transmission rate remains unchanged but the death rate increased
3) both the death rate and the transmission rate is increased.

While this is noteworthy, we will note dive too much into the semantics of the word "deadly" and just consider the what happens when we increase the death rate. As seen in earlier presented results, the corrupted blood disease is already deadly enough. On avreage the entire population is extinguished. however, intrestingly, the threshold at which the disease kills too fast for it to spread is seen in the below

For our system, our death rate is expressed as μ. With $S \approx N$ early on, $R_0 = \dfrac{\beta}{\gamma + \mu}$. Increasing μ makes the disease more lethal per case, but it also removes infected players from the population faster, which lowers the reproduction number R_0. So there should we expect there to be a threshold:

$$\mu^\star = \beta - \gamma$$

beyond which the disease can no longer sustain an epidemic at all. A deadlier disease that kills (or drives off) its hosts too quickly, will burns out before it can spread. This is the textbook "virulence/transmission trade-off" from evolutionary epidemiology. We contruct a series of mu values between 0 and 1, and for each mu value we simulate 120 runs. from a collection of runs with a given mu value, we extract the average population fraction that died/logged off.

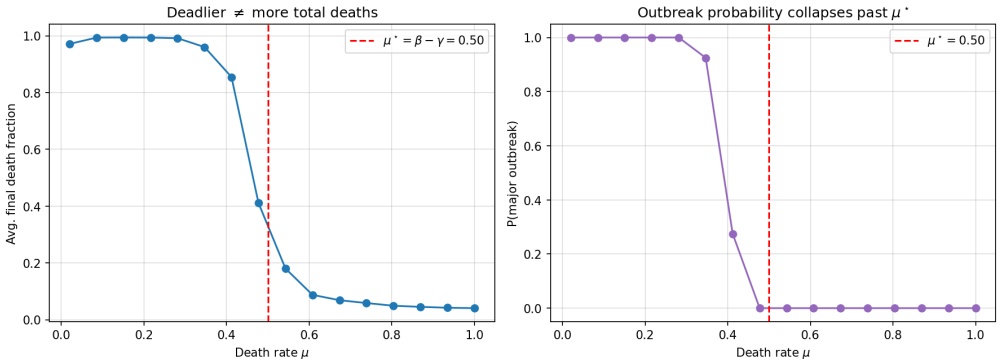


Figure 5: Deadliness analysis of different μ values

As expected, for increasing degree of deadliness, the ability for a disease to reproduce itself drops significantly. As μ grows past $\mu^\star = \beta - \gamma = 0.5$ (with $\beta = 0.8,\ \gamma = 0.3$), both the probability of a major outbreak and the expected total death toll collapse, because infected players are removed from the pool of potential spreaders faster than they can pass the disease on. The Corrupted Blood parameters used in the report ($\mu = 0.15 \ll \mu^\star$) sit deep in the regime where the disease is not deadly enough (per individual) to prevent it from wiping out almost everyone. This is exactly the near-total-extinction result in Table 1. A disease can only extinguish the whole population if it is contagious enough, relative to how quickly it removes its hosts, to keep $R_0 > 1$ all the way down to low susceptible counts.

1.4 d: For which population sizes and parameter values are deterministic models based on differential equations of sufficient precision

Deterministic models (ODEs) rely on the Law of Large Numbers and represent the expected average behavior of a system. They are of sufficient precision when:

N is large: Demographic stochasticity (random noise) becomes negligible compared to the total population.
I_0 is large: It guarantees the outbreak escapes the initial phase of stochastic extinction (as explored in the earlier task).For small populations (e.g., $N = 100$), discrete, random events heavily dictate the trajectory, often deviating wildly from the ODE predictions.
we can support these claims by running a series of simulation with increasingly larger values of N and I_0, but where the fraction $\frac{I_0}{N}$ remains the same to keep the ODE results consistent across all the variant simulations. We then also run our stochastic version, and compare;

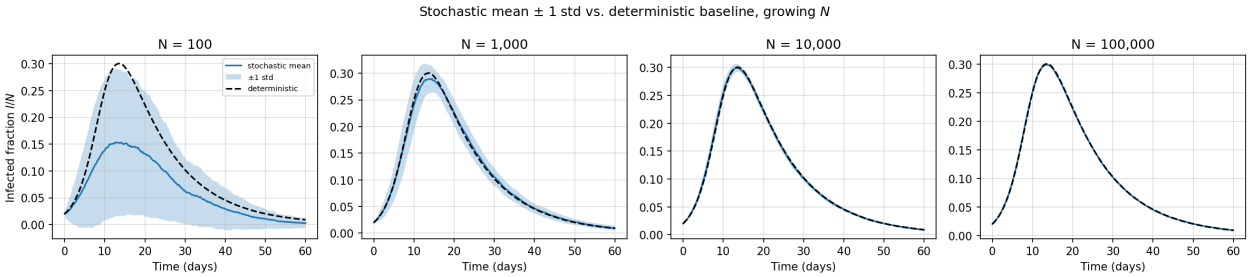


Figure 6: stochastic vs deterministic at different sizes of N

2 Part 2 - some ideas for further studies:

2.1 Include more states, like states where individuals are infected but don't yet show symptoms with different rates for contagion

In the baseline SISD model, a susceptible player becomes infected immediately after contact with an infected player. This means that the model only contains one infected state, where infected players are assumed to contribute directly to the spread of the disease. However, in this part, we include an intermediate and additional state for individuals who have been exposed to the disease, but who are not yet fully infected or do not yet show symptoms. To study this, we extend the SISD model by introducing an exposed state, denoted by E. Players in this state have been exposed to the infection, but have not yet progressed to the fully infected state I. This gives the following model structure

$$S \to E \to I, \qquad I \to S, \qquad I \to D$$

Here, susceptible players first move to the exposed state after contact with infectious players. Exposed players then progress to the infected state with rate α. As in the baseline model, infected players either recover and return to the susceptible state with rate γ, or leave the active population with rate μ. A reason for adding the exposed state is that exposed players may have a different levels of contagion than fully infected players. We therefore distinguish between the transmission rate from infected players, $\beta_I = 0.8$, and the transmission rate from exposed players, β_E. The infection rate is then given by

$$\lambda_{S\to E} = \frac{S(\beta_I I + \beta_E E)}{S + E + I}$$

The remaining transition rates are given as follows

$$\lambda_{E\to I} = \alpha E, \qquad \lambda_{I\to S} = \gamma I, \qquad \lambda_{I\to D} = \mu I$$

This extension allows us to investigate how hidden or early-stage infection affects the outbreak. In particular, we vary β_E to study how the contagiousness of exposed players influences the peak number of exposed and infected players, the final number of removed players, and the duration of the epidemic. With $\beta_E \in [0.0, 0.2, 0.4, 0.8]$ we achieve the results listed in Table 2

β_E	Mean peak I	Mean peak $E+I$	Mean final D	Mean time duration
0.0	1609.31	3021.79	9993.77	209.79
0.2	2122.32	4021.46	9996.18	176.43
0.4	2539.96	4850.90	9997.01	157.66
0.8	3132.32	6078.15	9997.96	140.48

Table 2: Effect of exposed-state transmission rate β_E on outbreak severity in the SEISD model.

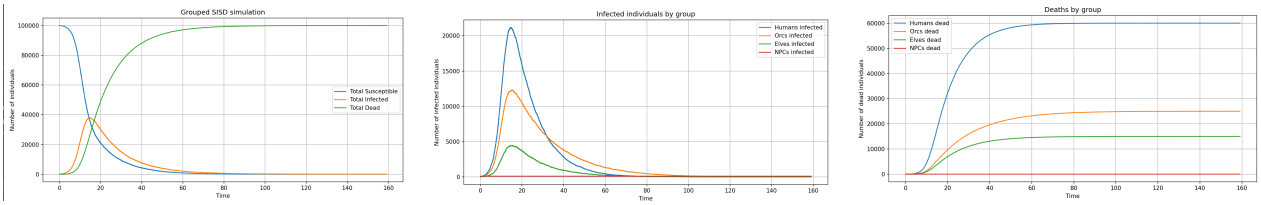
As seen in Table 2, low values of β_E lead to lower peaks for both infected players and the total number of exposed and infected players. However, the epidemic lasts longer on average. Conversely, higher values of β_E lead to larger peaks, but a shorter average duration of the epidemic. This suggests that when exposed players are more contagious, the disease spreads faster through the population and reaches a larger peak, but also burns out more quickly. The mean final number of removed players is close to the full population size for all tested values of β_E. Therefore, in this parameter setting, changing β_E does not strongly affect whether the outbreak becomes severe, since all simulations lead to a large outbreak. Instead, β_E mainly affects how quickly the outbreak develops and how high the peak becomes.

2.2 Model subgroups like families, workplaces and public transportation with different levels of exposure.

We added 4 different groups to the simulation. Humans which acts as a baseline with parameters $\beta = 0.8, \gamma = 0.3, \mu = 0.15$. Then we have Orcs which are strong, but dirty so we give them 50% lower death rate and 40% higher infection rate. The elves are really advanced healers so they have a 70% higher recovery rate and lastly we have the NPC. The NPC's doesn't have HP and thus they cannot die, but just like the actual bug they can get infected. And since these are in high trafic areas like shops and quests they infect others more. This was a bit tricky to implement because we only had β which controls how easily a group gets infected. So instead we set all the 100 NPC to be infected, without the possibility to change state. And the added a new infectiousness variable that contributes to the infection rate based on the number of NPC's.

Group	$\mathbf{S_0}$	$\mathbf{I_0}$	$\boldsymbol{\beta}$	$\boldsymbol{\gamma}$	$\boldsymbol{\mu}$	**Infectiousness**	**Active**
Humans	60000	20	0.800	0.300	0.143	1.0	True
Orcs	25000	0	1.120	0.300	0.071	1.2	True
Elves	15000	0	0.800	0.510	0.143	1.0	True
NPCs	0	100	0.000	0.000	0.000	1.5	False

Table 3: Configuration of the grouped SISD model.



(a) Trajectories (b) Infection by group (c) Death by group

Figure 7: Overall caption for the three images

2.3 Model spatial distribution of persons, including moving patterns that depend on the state.

To model a spatial newtwork we have transformed our model into a metapopulation model: we configured a network topology consisting of $K = 5$ distinct cities: [Ironforge, Stormwind, Ogrimar, Undercity, Thunder Bluff]. These cities are represented as nodes connected by Undirected transportation routes, i.e flightpaths or teleportation, represented by edges/arrows. The city of Orgrimmar was designated as the central high-traffic hub node, directly connected to four peripheral other cities.

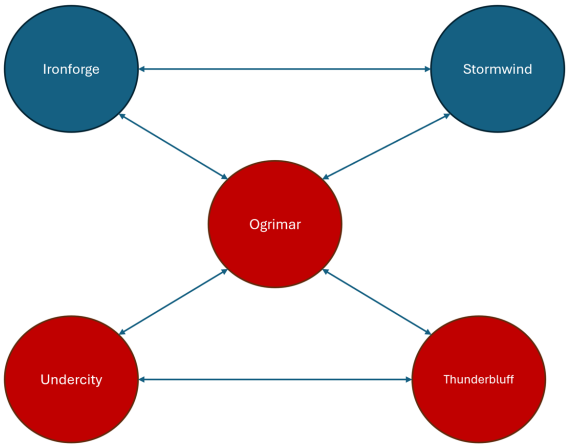


Figure 8: City Connectivity network

To introduce behavioral realism, transport rates rates were modeled as state dependent. Meaning that the travel rates would be different between healthy, sick and dead. healthy individuals travel at a baseline mmigration rate of θ_S, while infected individuals travel at a reduced rate on the form: θ_I = travel factor I where travel factor I $\in [0, 1]$ parameterized the impact of the disease on mobility. Scaling continuous time markov chain to spatial networks introduces exponentially high variable event space as, as the number of cities/routes increases. Rather than hardcoding seperate state-transitions for every possible combination of cities and route, our python implementation dynamically structured a flat propensity vector α containing all competing events across the global system. The global propensity vector α is systematically partitioned into local and spatial event blocks. For each city $k \in \{1, \ldots, K\}$ and route $e \in \{1, \ldots, E\}$, the propensities are aligned as follows:

$$\begin{aligned}
\text{Local Infections } (K \text{ events}): \quad & \alpha_k^{\text{infec}} = \beta \frac{S_k I_k}{N_k} \\
\text{Local Recoveries } (K \text{ events}): \quad & \alpha_k^{\text{recov}} = \gamma I_k \\
\text{Local Deaths } (K \text{ events}): \quad & \alpha_k^{\text{death}} = \mu I_k \\
\text{Susceptible Travel } (E \text{ events}): \quad & \alpha_e^{\text{trav}_S} = \theta_S S_{\text{origin}} \\
\text{Infected Travel } (E \text{ events}): \quad & \alpha_e^{\text{trav}_I} = \theta_I I_{\text{origin}}
\end{aligned}$$

The total global event rate A is subsequently updated at each discrete event step:

$$A = \sum_i \alpha_i$$

The continuous-time Markov chain is advanced dynamically by generating two independent random variables. First, the waiting time Δt until the next network transition is distributed exponentially according to the total system activity:

$$\Delta t \sim \text{Exp}(A) \tag{1}$$

Second, conditional on an event occurring at $t + \Delta t$, the specific transition index j is sampled according to the discrete probability distribution:

$$P(\text{event } j) = \frac{\alpha_j}{A} \tag{2}$$

A dedicated decoding block translates chosen index j back into its corresponding action and spatial location. For instance, if j falls within the travel segments the engine decrements the respective compartment (S or I) in the source city and increments it in the destination city, dynamically updating the local population denominators N_k for subsequent propensity calculations.

simulation configurations: The network was initialized with the total population distributed across the nodes, placing a minor initial infection load $I_0 = 20$ exclusively in Orgrimmar (the central node), to emmulate the initial spread of the Corrputed blood incident (people would cue up for raids in cities, once the raid was done they would re-appear in major cities again). We initilized the model with a starting ppulation with

city	Org	IF	SW	UC	TB
startin population	600	400	400	300	300

We established the following baseline travel pattern matrice for the cities:

	Org	IF	SW	UC	TB
Org	0	0.02	0.02	0.01	0.01
IF	0.02	0	0.01	0	0
SW	0.02	0.01	0	0	0
UC	0.01	0	0	0	0.01
TB	0.01	0	0	0.01	0

thus defining our travel rate matrix as;

$$M_{travel} = \begin{bmatrix} 0 & 0.02 & 0.02 & 0.01 & 0.01 \\ 0.02 & 0 & 0.01 & 0 & 0 \\ 0.02 & 0.01 & 0 & 0 & 0 \\ 0.01 & 0 & 0 & 0 & 0.01 \\ 0.01 & 0 & 0 & 0.01 & 0 \end{bmatrix}$$

We are also specifically tasked with exploring how moving patterns depend on the state. This is where we utilize the "travel factor I" argument in our python implementation. It functions as a multiplier of infected-player travel rate, relative to what the normal travel rate is of a susceptible person. We initilize our model with 3 different scenarios;
`travel_factor_I` $= 0.1$: sick players are weakend and mostly stay where they are.
`travel_factor_I` $= 1$: sick players disregard their health and move around as normal, potentially spreading more disease.
`travel_factor_I` $= 3$: panicked players flee an infected city at high rate, with 3 times the normal travel rate. Arguably the most historically accurate case, since players would just teleport away from the outbreak site the moment they cought notice of it.
We utilized the "Networkx" module in python, a common tool to handle these sort of node-to-node operational tasks. Our python implementation gave the following outputs:

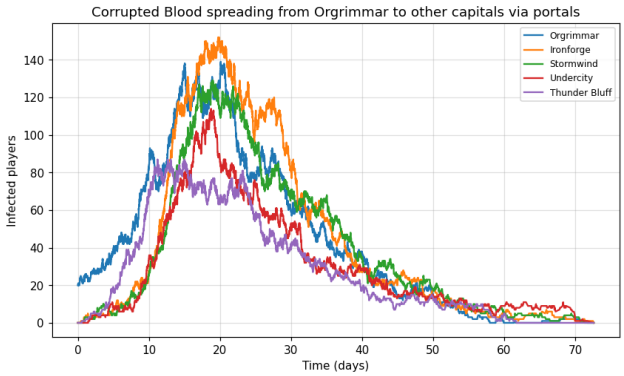


Figure 9: cumulative infections across all cities

We likewise also tracked the number of infected outside of Orgrimmar (i.e, everyone who is not in the city where the outbreak started), and the travel speed of those players. This is a strong indicator of the spatial distance the different type of carriers might have contributed to the spread of the disease.

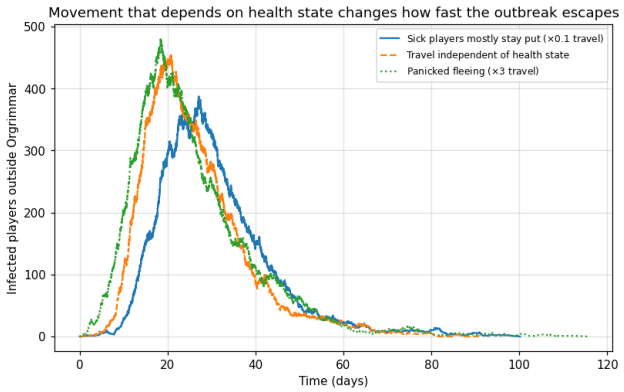


Figure 10: disease movement analysis in relation to travel speed

2.4 Model the effect of vaccination for different effectiveness of immunity potions.

As the disease spreads througout the realm, the elves use their advaced healing capabilities to develop an imunity potion.

To study the preventiveness of an immunity potion we extend the SISD model with a *vaccinated* state V. Susceptible are vaccinated at rate ν ($S \to V$). The vaccine is *leaky*: a vaccinated player is still infectable but at reduced rate $\beta(1-\text{eff})$, where $\text{eff} \in [0,1]$ ($\text{eff} = 1$ full immunity, $\text{eff} = 0$ useless). The simulation uses the same CTMC scheme with two infection channels (S and V) plus a vaccination event. Using the parameters from part 1: ($\beta = 0.8, \gamma = 0.3, \mu = 0.15$), we sweep effectiveness eff and rollout speed ν, reporting mean total deaths over 400 replicates with 95% confidence intervals.

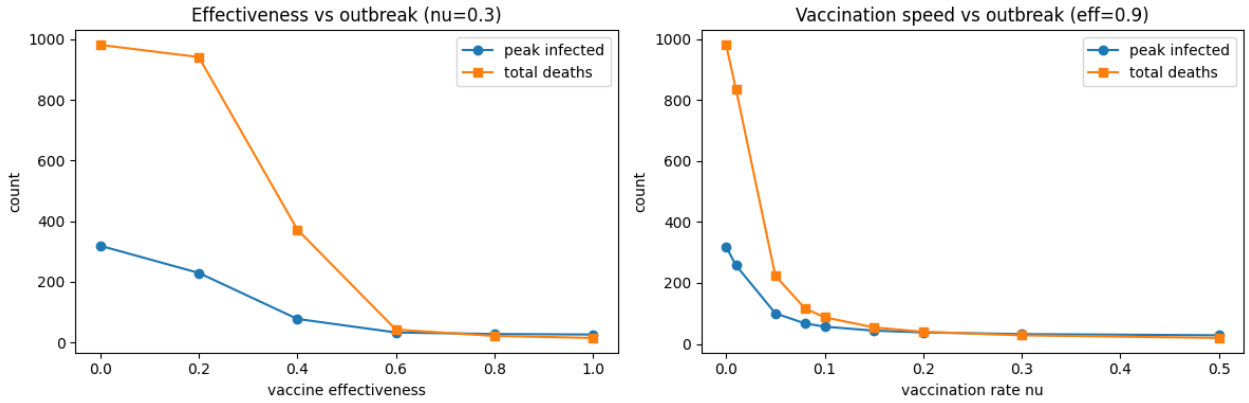


Figure 11: Outbreak size versus vaccine effectiveness (left) and vaccination speed ν (right).

The response is a threshold (group-immunity) effect, not linear: weak or slow vaccination barely changes the death toll, but once the campaign is strong enough to push the effective reproduction number below one, the outbreak collapses abruptly. (Note: recovered players return to S regardless of prior vaccination, so the V pool slowly erodes; this has little effect on a single-wave death toll.

Each plotted value is a mean over $n = 500$ stochastic runs, so we report a 95% confidence interval $\bar{D} \pm 1.96\,\hat{\sigma}/\sqrt{n}$ (Table 4). The intervals are tight everywhere except at the threshold eff $= 0.4$, where the standard deviation jumps to 312. This is not mere noise: at the threshold the outcome is bimodal, runs either fizzle out or grow into a near-total epidemic, so the mean (693) falls in the gap between the two modes and describes almost no individual run. Near the threshold the variance is itself the finding.

eff	mean	std	95% CI
0.0	994.4	4.4	±0.38
0.2	988.0	44.3	±3.89
0.4	692.6	312.1	±27.36
0.6	43.1	19.1	±1.67
0.8	21.6	6.7	±0.59

Table 4: Mean total deaths with 95% CI ($\nu = 0.6$, $n = 500$).

References

[red] reddit. *How blizzard coped with world of warcrafts blood*. URL: https://www.reddit.com/r/Games/comments/5hwkzz/how_blizzard_coped_with_world_of_warcrafts_blood/.

[Wika] Wikipedia. *Corrupted Blood incident*. URL: https://en.wikipedia.org/wiki/Corrupted_Blood_incident.

[Wikb] Wikipedia. *Exponential distribution*. URL: https://en.wikipedia.org/wiki/Exponential_distribution.

[Wowa] Wowhead. *Fresh 70 gear*. URL: https://www.wowhead.com/tbc/gear-set/fresh-70-gear-137813.

[Wowb] Wowpedia. *Corrupted Blood*. URL: https://wowpedia.fandom.com/wiki/Corrupted_Blood_(debuff).

.1 Task 0

```python
import numpy as np
import matplotlib.pyplot as plt
from scipy.integrate import solve_ivp


#################### ------------------- SISD ODE Model ------------------- ####################

def sisd_ode(t, y, beta, gamma, mu):
    """
    Deterministic SISD model using differential equations.

    Compartments
    ------------
    S : Susceptible individuals
    I : Infected individuals
    D : Dead individuals

    Transitions
    -----------
    S -> I : infection
    I -> S : recovery
    I -> D : death
    """
    S, I, D = y

    # Alive population
    N = S + I

    # Avoid division by zero
    if N <= 0:
        infection_rate = 0
    else:
        infection_rate = beta * S * I / N

    recovery_rate = gamma * I
    death_rate = mu * I

    dSdt = -infection_rate + recovery_rate
    dIdt = infection_rate - recovery_rate - death_rate
    dDdt = death_rate

    return [dSdt, dIdt, dDdt]


def simulate_sisd_ode(N, I0, beta, gamma, mu, t_max=160):
    """
    Solve the SISD differential equation model.
    """
    S0 = N - I0
    D0 = 0

    y0 = [S0, I0, D0]

    t_eval = np.linspace(0, t_max, 1000)

    solution = solve_ivp(
        fun=sisd_ode,
```

```
        t_span=(0, t_max),
        y0=y0,
        args=(beta, gamma, mu),
        t_eval=t_eval
    )

    return solution.t, solution.y.T


def plot_sisd_ode(times, states, title="Deterministic SISD model"):
    """
    Plot S, I and D over time.
    """
    S = states[:, 0]
    I = states[:, 1]
    D = states[:, 2]

    plt.figure(figsize=(10, 5))

    plt.plot(times, S, label="Susceptible")
    plt.plot(times, I, label="Infected")
    plt.plot(times, D, label="Dead")

    plt.xlabel("Time")
    plt.ylabel("Number of individuals")
    plt.title(title)
    plt.legend()
    plt.grid(True)

    plt.tight_layout()
    plt.show()


def main():
    # Population settings
    N = 1_000_000
    I0 = 20

    # Disease parameters
    beta = 0.7
    gamma = 0.3
    mu = 0.15

    t_max = 160

    times, states = simulate_sisd_ode(
        N=N,
        I0=I0,
        beta=beta,
        gamma=gamma,
        mu=mu,
        t_max=t_max
    )

    plot_sisd_ode(
        times,
        states,
        title="Deterministic SISD model"
    )

```

```
    S_final = states[-1, 0]
    I_final = states[-1, 1]
    D_final = states[-1, 2]

    print("Final values")
    print("------------")
    print(f"Susceptible: {S_final:.2f}")
    print(f"Infected:    {I_final:.2f}")
    print(f"Dead:        {D_final:.2f}")


if __name__ == "__main__":
    main()

```

.2 Task 1a

```
import numpy as np
import matplotlib.pyplot as plt


#################### -------------------- SISD Simulation -------------------- ####################

def simulate_sisd(N, I0, beta, gamma, mu, t_max=np.inf):
    """
    Simulate a stochastic SIS model with death using the Gillespie algorithm.

    Compartments
    ------------
    S : Susceptible players
    I : Infected players
    D : Dead players

    Events
    ------
    Infection: S + I -> I + I
    Recovery: I -> S
    Death:    I -> D

    Parameters
    ----------
    N : int
        Initial total population size.
    I0 : int
        Initial number of infected individuals.
    beta : float
        Infection rate.
    gamma : float
        Recovery rate.
    mu : float
        Death rate.
    t_max : float
        Maximum simulation time.

    Returns
    -------
    times : np.array
        Times at which events occur.
```

```
    states : np.array
        Array with columns [S, I, D].
    events : list
        List of event types.
    """
    S = N - I0
    I = I0
    D = 0
    t = 0.0

    times = [t]
    states = [[S, I, D]]
    events = []

    while I > 0 and t < t_max:
        N_alive = S + I

        if N_alive <= 0:
            break

        # Infection: S + I -> I + I
        infection_rate = beta * S * I / N_alive

        # Recovery: I -> S
        recovery_rate = gamma * I

        # Death: I -> D
        death_rate = mu * I

        total_rate = infection_rate + recovery_rate + death_rate

        if total_rate <= 0:
            break

        # Time until next event
        dt = np.random.exponential(1 / total_rate)
        t += dt

        if t > t_max:
            break

        # Decide event type
        u = np.random.rand()

        # print("u", u)
        # print("dt", dt)
        # print("infection", infection_rate / total_rate)
        # print("recovery", (recovery_rate) / total_rate)
        # print("Death", 1 - (infection_rate + recovery_rate) / total_rate)


        if u < infection_rate / total_rate:
            S -= 1
            I += 1
            events.append("infection")

        elif u < (infection_rate + recovery_rate) / total_rate:
            I -= 1
            S += 1
            events.append("recovery")
```

```
        else:
            I -= 1
            D += 1
            events.append("death")

        times.append(t)
        states.append([S, I, D])

    return np.array(times), np.array(states), events


def plot_sisd(times, states, title="Stochastic SISD simulation"):
    """
    Plot S, I and D over time.
    """
    S = states[:, 0]
    I = states[:, 1]
    D = states[:, 2]

    plt.figure(figsize=(10, 5))
    plt.plot(times, S, label="Susceptible")
    plt.plot(times, I, label="Infected")
    plt.plot(times, D, label="Dead")

    plt.xlabel("Time")
    plt.ylabel("Number of individuals")
    plt.title(title)
    plt.legend()
    plt.grid(True)

    plt.tight_layout()
    plt.savefig("assets_test/sisd_1.png")

def run_many_simulations(N, I0, beta, gamma, mu, n_simulations,
↪   major_outbreak_threshold=0.1):
    """
    Run many stochastic SISD simulations and collect summary statistics.
    """
    final_dead = []
    peak_infected = []
    epidemic_durations = []
    major_outbreaks = []

    for _ in range(n_simulations):
        print("Running simulation for summary", _)
        times, states, events = simulate_sisd(N, I0, beta, gamma, mu)

        S = states[:, 0]
        I = states[:, 1]
        D = states[:, 2]

        final_D = D[-1]
        peak_I = np.max(I)
        duration = times[-1]

        final_dead.append(final_D)
        peak_infected.append(peak_I)
        epidemic_durations.append(duration)

```

```
        major_outbreaks.append(peak_I > major_outbreak_threshold * N)

    results = {
        "final_dead": np.array(final_dead),
        "peak_infected": np.array(peak_infected),
        "epidemic_durations": np.array(epidemic_durations),
        "major_outbreaks": np.array(major_outbreaks)
    }

    return results

def print_summary(results, N):
    """
    Print summary statistics from many SISD simulations.
    """
    final_dead = results["final_dead"]
    peak_infected = results["peak_infected"]
    epidemic_durations = results["epidemic_durations"]
    major_outbreaks = results["major_outbreaks"]

    print("Summary of stochastic SISD simulations")
    print("--------------------------------------")
    print(f"Average final deaths: {np.mean(final_dead):.2f}")
    print(f"Average final death fraction: {np.mean(final_dead) / N:.4f}")
    print(f"Average peak infected: {np.mean(peak_infected):.2f}")
    print(f"Average epidemic duration: {np.mean(epidemic_durations):.2f}")
    print(f"Probability of major outbreak: {np.mean(major_outbreaks):.4f}")
    print(f"Probability of early extinction: {1 - np.mean(major_outbreaks):.4f}")

def plot_final_death_histogram(results):
    """
    Plot histogram of final number of deaths.
    """
    final_dead = results["final_dead"]

    plt.figure(figsize=(8, 5))
    plt.hist(final_dead, bins=30, edgecolor="black")

    plt.xlabel("Final number of dead individuals")
    plt.ylabel("Frequency")
    plt.title("Distribution of final deaths")
    plt.grid(True)

    plt.tight_layout()
    plt.savefig("assets_test/final_deaths.png")


def plot_peak_infected_histogram(results):
    """
    Plot histogram of peak infected individuals.
    """
    peak_infected = results["peak_infected"]

    plt.figure(figsize=(8, 5))
    plt.hist(peak_infected, bins=30, edgecolor="black")

    plt.xlabel("Peak number of infected individuals")
    plt.ylabel("Frequency")
    plt.title("Distribution of peak infected")
    plt.grid(True)
```

```
    plt.tight_layout()
    plt.savefig("assets_test/peak_infections.png")

def plot_many_sisd_trajectories(N, I0, beta, gamma, mu, n_simulations, t_max=160):
    """
    Plot S, I and D trajectories from many stochastic SISD simulations.
    """
    plt.figure(figsize=(10, 6))

    # Fixed colors for each compartment
    colors = {
        "S": "tab:blue",
        "I": "tab:orange",
        "D": "tab:green"
    }

    for _ in range(n_simulations):
        print("Runing sim for plot", _)
        times, states, events = simulate_sisd(
            N=N,
            I0=I0,
            beta=beta,
            gamma=gamma,
            mu=mu,
            t_max=t_max
        )

        S = states[:, 0]
        I = states[:, 1]
        D = states[:, 2]

        plt.plot(times, S, color=colors["S"], alpha=0.15, linewidth=1)
        plt.plot(times, I, color=colors["I"], alpha=0.15, linewidth=1)
        plt.plot(times, D, color=colors["D"], alpha=0.15, linewidth=1)

    plt.xlabel("Time")
    plt.ylabel("Number of individuals")
    plt.title("Stochastic SISD trajectories")
    plt.grid(True, linestyle="--", alpha=0.5)

    # Dummy plots for legend with matching colors
    plt.plot([], [], color=colors["S"], label="Susceptible")
    plt.plot([], [], color=colors["I"], label="Infected")
    plt.plot([], [], color=colors["D"], label="Dead")
    plt.legend()

    plt.tight_layout()
    plt.savefig("assets_test/sisd_trajectories.png")
    plt.show()

def main():
    np.random.seed(42)

    # Population settings
    N = 100000
    I0 = 20

    # Disease parameters
    beta = 0.8        # Infection rate
```

```
    gamma = 0.3     # Recovery rate
    mu = 1 / 7      # Death rate

    # One simulation
    times, states, events = simulate_sisd(
        N=N,
        I0=I0,
        beta=beta,
        gamma=gamma,
        mu=mu
    )

    plot_sisd(
        times,
        states,
        title="Stochastic SISD simulation"
    )

    # Many simulations
    n_simulations = 100

    results = run_many_simulations(
        N=N,
        I0=I0,
        beta=beta,
        gamma=gamma,
        mu=mu,
        n_simulations=n_simulations
    )

    print_summary(results, N)

    plot_final_death_histogram(results)
    plot_peak_infected_histogram(results)

    plot_many_sisd_trajectories(
        N=N,
        I0=I0,
        beta=beta,
        gamma=gamma,
        mu=mu,
        n_simulations=100,
        t_max=160
    )


if __name__ == "__main__":
    main()
```

.3 Task 1b

.4 Task 1c

```


def simulate_sisd(N, I0, beta, gamma, mu, t_max=np.inf, rng=None):
    """
    Stochastic SISD model via the Gillespie algorithm (fixed version of code_1.py).

```

```
    Returns
    -------
    times  : np.array of event times (including t=0 and, if t_max is finite, t_max)
    states : np.array with columns [S, I, D]
    events : list of event-type strings
    """
    rng = np.random.default_rng() if rng is None else rng
    S, I, D = N - I0, I0, 0
    t = 0.0
    times, states, events = [t], [[S, I, D]], []

    while I > 0 and t < t_max:
        N_alive = S + I  # dead players no longer take part in contacts
        infection_rate = beta * S * I / N_alive
        recovery_rate = gamma * I
        death_rate = mu * I
        total_rate = infection_rate + recovery_rate + death_rate

        if total_rate <= 0:
            break

        dt = rng.exponential(1.0 / total_rate)

        if t + dt > t_max:          # censor: hold the last state at t_max (fix #2)
            times.append(t_max)
            states.append([S, I, D])
            break

        t += dt
        u = rng.random()
        if u < infection_rate / total_rate:
            S -= 1; I += 1; events.append("infection")
        elif u < (infection_rate + recovery_rate) / total_rate:
            I -= 1; S += 1; events.append("recovery")
        else:
            I -= 1; D += 1; events.append("death")

        times.append(t)
        states.append([S, I, D])

    return np.array(times), np.array(states, dtype=float), events


def simulate_sisd_deterministic(N, I0, beta, gamma, mu, dt=0.05, t_max=160):
    """
    Deterministic discrete-time (Euler-discretised ODE) baseline.
    Used ONLY for comparison (Part 0 / Part 1d) | never as the main model.
    """
    n_steps = int(t_max / dt)
    S = np.empty(n_steps + 1); I = np.empty(n_steps + 1); D = np.empty(n_steps + 1)
    S[0], I[0], D[0] = N - I0, I0, 0.0

    for k in range(n_steps):
        Sk, Ik = S[k], I[k]
        N_alive = max(Sk + Ik, 1e-9)
        dS = -beta * Ik * Sk / N_alive + gamma * Ik   # recovery returns to S (no
        ↪  immunity)
        dI = beta * Ik * Sk / N_alive - gamma * Ik - mu * Ik
        dD = mu * Ik
        S[k + 1] = Sk + dt * dS
```

```
        I[k + 1] = max(Ik + dt * dI, 0.0)
        D[k + 1] = D[k] + dt * dD

    t_grid = np.linspace(0, t_max, n_steps + 1)
    return t_grid, np.column_stack([S, I, D])


def run_many_simulations(N, I0, beta, gamma, mu, n_simulations,
↪   major_outbreak_threshold=0.1,
                         t_max=np.inf, rng=None):
    """Run many stochastic SISD simulations and collect summary statistics.

    `t_max` is a safety valve: for mu (and gamma) very small relative to beta, the
    model is essentially a supercritical SIS process whose *true* extinction time
    can be astronomically long (classic quasi-stationary persistence). Capping
    t_max avoids runaway memory/time use in that regime; runs that hit the cap are
    simply treated as "still ongoing" snapshots rather than true endpoints.
    """
    rng = np.random.default_rng() if rng is None else rng
    final_dead, peak_infected, durations, major = [], [], [], []

    for _ in range(n_simulations):
        times, states, _ = simulate_sisd(N, I0, beta, gamma, mu, t_max=t_max, rng=rng)
        final_dead.append(states[-1, 2])
        peak_infected.append(states[:, 1].max())
        durations.append(times[-1])
        major.append(states[:, 1].max() > major_outbreak_threshold * N)

    return {
        "final_dead": np.array(final_dead),
        "peak_infected": np.array(peak_infected),
        "epidemic_durations": np.array(durations),
        "major_outbreaks": np.array(major),
    }


def print_summary(results, N):
    fd, pk, du, mo = (results["final_dead"], results["peak_infected"],
                      results["epidemic_durations"], results["major_outbreaks"])
    print("Summary of stochastic SISD simulations")
    print("-" * 40)
    print(f"Average final deaths:          {fd.mean():.2f}")
    print(f"Average final death fraction:  {fd.mean()/N:.4f}")
    print(f"Average peak infected:         {pk.mean():.2f}")
    print(f"Average epidemic duration:     {du.mean():.2f}")
    print(f"P(major outbreak):             {mo.mean():.4f}")
    print(f"P(early extinction):            {1 - mo.mean():.4f}")




mu_values = np.linspace(0.02, 1.0, 16)    # avoid mu=0 exactly: that's a supercritical
↪   SIS
                                          # process with no death-driven removal at all,
                                          ↪   and its
                                          # true extinction time can be astronomically
                                          ↪   long (see
                                          # discussion above) | t_max below guards
                                          ↪   against this
```

```
                                        # for any mu close to the danger zone too.
N_1c, n_sims_1c, I0_1c, t_max_1c = 1000, 120, 20, 300
mu_star = BETA - GAMMA

rng = np.random.default_rng(30)
death_fraction, p_major = [], []
for mu in mu_values:
    res = run_many_simulations(N_1c, I0_1c, BETA, GAMMA, mu, n_simulations=n_sims_1c,
                               t_max=t_max_1c, rng=rng)
    death_fraction.append(res["final_dead"].mean() / N_1c)
    p_major.append(res["major_outbreaks"].mean())

fig, axes = plt.subplots(1, 2, figsize=(12, 4.5))
axes[0].plot(mu_values, death_fraction, "o-")
axes[0].axvline(mu_star, color="red", linestyle="--",
↪  label=f"$\\mu^\\star=\\beta-\\gamma={mu_star:.2f}$")
axes[0].set_xlabel("Death rate $\\mu$"); axes[0].set_ylabel("Avg. final death fraction")
axes[0].set_title("Deadlier $\\neq$ more total deaths"); axes[0].legend()

axes[1].plot(mu_values, p_major, "o-", color="tab:purple")
axes[1].axvline(mu_star, color="red", linestyle="--",
↪  label=f"$\\mu^\\star={mu_star:.2f}$")
axes[1].set_xlabel("Death rate $\\mu$"); axes[1].set_ylabel("P(major outbreak)")
axes[1].set_title("Outbreak probability collapses past $\\mu^\\star$"); axes[1].legend()
fig.tight_layout()
savefig(fig, "part1c_mu_sweep.png")
plt.show()
```

.5 Task 1d

```


N_list = [100, 1_000, 10_000, 100_000]
reps_list = [300, 150, 40, 10]
I0_frac = 0.02
t_max_1d = 60
t_grid_1d = np.linspace(0, t_max_1d, 300)

rng = np.random.default_rng(40)
results_1d = {}
for N, reps in zip(N_list, reps_list):
    I0 = max(1, round(I0_frac * N))
    I_paths = np.empty((reps, len(t_grid_1d)))
    for r in range(reps):
        t_r, st_r, _ = simulate_sisd(N, I0, BETA, GAMMA, MU, t_max=t_max_1d, rng=rng)
        I_paths[r] = step_interp(t_r, st_r[:, 1], t_grid_1d)
    t_det_N, st_det_N = simulate_sisd_deterministic(N, I0, BETA, GAMMA, MU,
    ↪  t_max=t_max_1d)
    I_det_N = step_interp(t_det_N, st_det_N[:, 1], t_grid_1d)
    results_1d[N] = dict(I_mean=I_paths.mean(axis=0), I_std=I_paths.std(axis=0),
                         I_det=I_det_N, I_paths=I_paths)


fig, axes = plt.subplots(1, len(N_list), figsize=(16, 3.6), sharex=True)
for ax, N in zip(axes, N_list):
```

```
    d = results_1d[N]
    frac_mean = d["I_mean"] / N
    frac_std = d["I_std"] / N
    ax.plot(t_grid_1d, frac_mean, label="stochastic mean")
    ax.fill_between(t_grid_1d, frac_mean - frac_std, frac_mean + frac_std, alpha=0.25,
    ↪  label="$\\pm 1$ std")
    ax.plot(t_grid_1d, d["I_det"] / N, "k--", label="deterministic")
    ax.set_title(f"N = {N:,}")
    ax.set_xlabel("Time (days)")
axes[0].set_ylabel("Infected fraction $I/N$")
axes[0].legend(fontsize=7)
fig.suptitle("Stochastic mean $\\pm$ 1 std vs. deterministic baseline, growing $N$")
fig.tight_layout()
savefig(fig, "part1d_convergence.png")
plt.show()


```

.6 Task 2a

```
def summarize_seisd_scenario(
    N, I0, E0, beta_I, beta_E, alpha, gamma, mu,
    n_simulations=100,
    major_outbreak_threshold=0.10
):
    peak_I_values = []
    peak_EI_values = []
    final_D_values = []
    duration_values = []
    major_outbreaks = []
    disappearances = []

    for _ in range(n_simulations):
        times, states, events = simulate_seisd(
            N=N,
            I0=I0,
            E0=E0,
            beta_I=beta_I,
            beta_E=beta_E,
            alpha=alpha,
            gamma=gamma,
            mu=mu
        )

        S = states[:, 0]
        E = states[:, 1]
        I = states[:, 2]
        D = states[:, 3]

        peak_I = np.max(I)
        peak_EI = np.max(E + I)
        final_D = D[-1]
        duration = times[-1]

        # Disease has disappeared if no exposed or infected remain
        disappeared = (E[-1] + I[-1]) == 0

        # Major outbreak if more than 10% are exposed/infected at peak
        major_outbreak = peak_EI > major_outbreak_threshold * N
```

```
        peak_I_values.append(peak_I)
        peak_EI_values.append(peak_EI)
        final_D_values.append(final_D)
        duration_values.append(duration)
        major_outbreaks.append(major_outbreak)
        disappearances.append(disappeared)

    # Confidence intervals
    p_major, p_major_low, p_major_high = proportion_ci(major_outbreaks)
    p_disappear, p_disappear_low, p_disappear_high = proportion_ci(disappearances)

    mean_peak_I, peak_I_low, peak_I_high = mean_ci(peak_I_values)
    mean_peak_EI, peak_EI_low, peak_EI_high = mean_ci(peak_EI_values)
    mean_final_D, final_D_low, final_D_high = mean_ci(final_D_values)
    mean_duration, duration_low, duration_high = mean_ci(duration_values)

    return {
        "beta_E": beta_E,
        "alpha": alpha,

        "p_major": p_major,
        "p_major_low": p_major_low,
        "p_major_high": p_major_high,

        "p_disappear": p_disappear,
        "p_disappear_low": p_disappear_low,
        "p_disappear_high": p_disappear_high,

        "mean_peak_I": mean_peak_I,
        "peak_I_low": peak_I_low,
        "peak_I_high": peak_I_high,

        "mean_peak_EI": mean_peak_EI,
        "peak_EI_low": peak_EI_low,
        "peak_EI_high": peak_EI_high,

        "mean_final_D": mean_final_D,
        "final_D_low": final_D_low,
        "final_D_high": final_D_high,

        "mean_duration": mean_duration,
        "duration_low": duration_low,
        "duration_high": duration_high,
    }

#  95% confidence interval for a mean
def mean_ci(x):
    x = np.array(x)
    mean = np.mean(x)
    se = np.std(x, ddof=1) / np.sqrt(len(x))
    lower = mean - 1.96 * se
    upper = mean + 1.96 * se
    return mean, lower, upper

# 95% confidence interval for a probability/proportion
def proportion_ci(indicators):
    indicators = np.array(indicators)
    n = len(indicators)
    p_hat = np.mean(indicators)
```

```
    se = np.sqrt(p_hat * (1 - p_hat) / n)
    lower = p_hat - 1.96 * se
    upper = p_hat + 1.96 * se
    return p_hat, lower, upper

def summarize_seisd_scenario(
    N, I0, E0, beta_I, beta_E, alpha, gamma, mu,
    n_simulations=100,
    major_outbreak_threshold=0.10
):
    peak_I_values = []
    peak_EI_values = []
    final_D_values = []
    duration_values = []
    major_outbreaks = []
    disappearances = []

    for _ in range(n_simulations):
        times, states, events = simulate_seisd(
            N=N,
            I0=I0,
            E0=E0,
            beta_I=beta_I,
            beta_E=beta_E,
            alpha=alpha,
            gamma=gamma,
            mu=mu
        )

        S = states[:, 0]
        E = states[:, 1]
        I = states[:, 2]
        D = states[:, 3]

        peak_I = np.max(I)
        peak_EI = np.max(E + I)
        final_D = D[-1]
        duration = times[-1]

        # Disease has disappeared if no exposed or infected remain
        disappeared = (E[-1] + I[-1]) == 0

        # Major outbreak if more than 10% are exposed/infected at peak
        major_outbreak = peak_EI > major_outbreak_threshold * N

        peak_I_values.append(peak_I)
        peak_EI_values.append(peak_EI)
        final_D_values.append(final_D)
        duration_values.append(duration)
        major_outbreaks.append(major_outbreak)
        disappearances.append(disappeared)

    # Confidence intervals
    p_major, p_major_low, p_major_high = proportion_ci(major_outbreaks)
    p_disappear, p_disappear_low, p_disappear_high = proportion_ci(disappearances)

    mean_peak_I, peak_I_low, peak_I_high = mean_ci(peak_I_values)
    mean_peak_EI, peak_EI_low, peak_EI_high = mean_ci(peak_EI_values)
    mean_final_D, final_D_low, final_D_high = mean_ci(final_D_values)
    mean_duration, duration_low, duration_high = mean_ci(duration_values)
```

```
    return {
        "beta_E": beta_E,
        "alpha": alpha,

        "p_major": p_major,
        "p_major_low": p_major_low,
        "p_major_high": p_major_high,

        "p_disappear": p_disappear,
        "p_disappear_low": p_disappear_low,
        "p_disappear_high": p_disappear_high,

        "mean_peak_I": mean_peak_I,
        "peak_I_low": peak_I_low,
        "peak_I_high": peak_I_high,

        "mean_peak_EI": mean_peak_EI,
        "peak_EI_low": peak_EI_low,
        "peak_EI_high": peak_EI_high,

        "mean_final_D": mean_final_D,
        "final_D_low": final_D_low,
        "final_D_high": final_D_high,

        "mean_duration": mean_duration,
        "duration_low": duration_low,
        "duration_high": duration_high,
    }

# Different exposed contagion rates
beta_E_values = [0.0, 0.2, 0.4, 0.8]

n_simulations = 100

rows = []

for beta_E in beta_E_values:
    result = summarize_seisd_scenario(
        N=N,
        I0=I0,
        E0=E0,
        beta_I=beta_I,
        beta_E=beta_E,
        alpha=alpha,
        gamma=gamma,
        mu=mu,
        n_simulations=n_simulations,
        major_outbreak_threshold=0.10
    )

    rows.append(result)

results_df = pd.DataFrame(rows)
results_df
```

.7 Task 2b

```python

import numpy as np
import matplotlib.pyplot as plt
import os


#################### ---------- Grouped SISD Simulation ---------- ####################

def simulate_grouped_sisd(groups, t_max=np.inf):
    """
    Simulate a stochastic SISD model with multiple population groups.

    Groups
    ------
    Humans : Standard rates
    Orcs   : Higher infection rate, lower death rate
    Elves  : Higher recovery rate
    NPCs   : Fixed infected reservoir. They cannot recover or die.

    Each non-NPC group has compartments:
    S : Susceptible
    I : Infected
    D : Dead
    """

    group_names = list(groups.keys())

    S = np.array([groups[g]["S0"] for g in group_names], dtype=int)
    I = np.array([groups[g]["I0"] for g in group_names], dtype=int)
    D = np.array([0 for _ in group_names], dtype=int)

    beta = np.array([groups[g]["beta"] for g in group_names], dtype=float)
    gamma = np.array([groups[g]["gamma"] for g in group_names], dtype=float)
    mu = np.array([groups[g]["mu"] for g in group_names], dtype=float)
    infectiousness = np.array([groups[g]["infectiousness"] for g in group_names], dtype=float)
    active = np.array([groups[g]["active"] for g in group_names], dtype=bool)

    t = 0.0

    times = [t]
    states = [(S.copy(), I.copy(), D.copy())]
    events = []

    while np.sum(I[active]) > 0 and t < t_max:
        N_alive = np.sum(S[active] + I[active])

        if N_alive <= 0:
            break

        # Total infection pressure from all infected groups.
        # NPCs contribute here even though they cannot change state.
        infection_pressure = np.sum(infectiousness * I)

        infection_rates = np.zeros(len(group_names))
        recovery_rates = np.zeros(len(group_names))
        death_rates = np.zeros(len(group_names))
```

```
        for k, g in enumerate(group_names):
            if active[k]:
                infection_rates[k] = beta[k] * S[k] * infection_pressure / N_alive
                recovery_rates[k] = gamma[k] * I[k]
                death_rates[k] = mu[k] * I[k]

        total_rate = (
            np.sum(infection_rates)
            + np.sum(recovery_rates)
            + np.sum(death_rates)
        )

        if total_rate <= 0:
            break

        # Time until next event
        dt = np.random.exponential(1 / total_rate)
        t += dt

        if t > t_max:
            break

        # Choose event
        rates = []
        labels = []

        for k, g in enumerate(group_names):
            if infection_rates[k] > 0:
                rates.append(infection_rates[k])
                labels.append(("infection", k))

            if recovery_rates[k] > 0:
                rates.append(recovery_rates[k])
                labels.append(("recovery", k))

            if death_rates[k] > 0:
                rates.append(death_rates[k])
                labels.append(("death", k))

        rates = np.array(rates)
        probabilities = rates / np.sum(rates)

        event_index = np.random.choice(len(labels), p=probabilities)
        event_type, k = labels[event_index]

        if event_type == "infection":
            S[k] -= 1
            I[k] += 1

        elif event_type == "recovery":
            I[k] -= 1
            S[k] += 1

        elif event_type == "death":
            I[k] -= 1
            D[k] += 1

        events.append((event_type, group_names[k]))

```

```
        times.append(t)
        states.append((S.copy(), I.copy(), D.copy()))

    return np.array(times), states, events, group_names


def unpack_states(states):
    """
    Convert list of state tuples into arrays:
    S, I, D each have shape: time x groups
    """
    S = np.array([state[0] for state in states])
    I = np.array([state[1] for state in states])
    D = np.array([state[2] for state in states])

    return S, I, D


#################### ---------- Plotting ---------- ####################

def plot_grouped_sisd(times, states, group_names, savepath="assets_test/grouped_sisd.png"):
    """
    Plot total S, I and D over time.
    """
    os.makedirs(os.path.dirname(savepath), exist_ok=True)

    S, I, D = unpack_states(states)

    total_S = np.sum(S, axis=1)
    total_I = np.sum(I, axis=1)
    total_D = np.sum(D, axis=1)

    plt.figure(figsize=(10, 5))
    plt.plot(times, total_S, label="Total Susceptible")
    plt.plot(times, total_I, label="Total Infected")
    plt.plot(times, total_D, label="Total Dead")

    plt.xlabel("Time")
    plt.ylabel("Number of individuals")
    plt.title("Grouped SISD simulation")
    plt.legend()
    plt.grid(True)

    plt.tight_layout()
    plt.savefig(savepath)
    plt.show()


def plot_group_infections(times, states, group_names, savepath="assets_test/group_infections.png"):
    """
    Plot infected individuals by group.
    """
    os.makedirs(os.path.dirname(savepath), exist_ok=True)

    S, I, D = unpack_states(states)

    plt.figure(figsize=(10, 5))
```

```
174
175     for k, group in enumerate(group_names):
176         plt.plot(times, I[:, k], label=f"{group} infected")
177
178     plt.xlabel("Time")
179     plt.ylabel("Number of infected individuals")
180     plt.title("Infected individuals by group")
181     plt.legend()
182     plt.grid(True)
183
184     plt.tight_layout()
185     plt.savefig(savepath)
186     plt.show()
187
188
189 def plot_group_deaths(times, states, group_names,
    ↪  savepath="assets_test/group_deaths.png"):
190     """
191     Plot dead individuals by group.
192     """
193     os.makedirs(os.path.dirname(savepath), exist_ok=True)
194
195     S, I, D = unpack_states(states)
196
197     plt.figure(figsize=(10, 5))
198
199     for k, group in enumerate(group_names):
200         plt.plot(times, D[:, k], label=f"{group} dead")
201
202     plt.xlabel("Time")
203     plt.ylabel("Number of dead individuals")
204     plt.title("Deaths by group")
205     plt.legend()
206     plt.grid(True)
207
208     plt.tight_layout()
209     plt.savefig(savepath)
210     plt.show()
211
212
213 ##################### -------------------- Summary --------------------
    ↪  ####################
214
215 def run_many_grouped_simulations(groups, n_simulations, t_max=np.inf):
216     """
217     Run many grouped SISD simulations and collect summary statistics.
218     """
219     final_dead = []
220     peak_infected = []
221     epidemic_durations = []
222
223     for i in range(n_simulations):
224         print("Running grouped simulation", i)
225
226         times, states, events, group_names = simulate_grouped_sisd(groups, t_max=t_max)
227
228         S, I, D = unpack_states(states)
229
230         total_I = np.sum(I, axis=1)
231         total_D = np.sum(D, axis=1)
```

```
232
233         final_dead.append(total_D[-1])
234         peak_infected.append(np.max(total_I))
235         epidemic_durations.append(times[-1])
236
237     results = {
238         "final_dead": np.array(final_dead),
239         "peak_infected": np.array(peak_infected),
240         "epidemic_durations": np.array(epidemic_durations)
241     }
242
243     return results
244
245
246 def print_grouped_summary(results, total_population):
247     """
248     Print summary statistics.
249     """
250     final_dead = results["final_dead"]
251     peak_infected = results["peak_infected"]
252     epidemic_durations = results["epidemic_durations"]
253
254     print("Summary of grouped stochastic SISD simulations")
255     print("-----------------------------------------------")
256     print(f"Average final deaths: {np.mean(final_dead):.2f}")
257     print(f"Average final death fraction: {np.mean(final_dead) / total_population:.4f}")
258     print(f"Average peak infected: {np.mean(peak_infected):.2f}")
259     print(f"Average epidemic duration: {np.mean(epidemic_durations):.2f}")
260
261
262 ##################### -------------------- Main -------------------- ####################
263
264 def main():
265     np.random.seed(42)
266
267     # Original standard disease parameters
268     beta = 0.8
269     gamma = 0.3
270     mu = 0.15
271
272     groups = {
273         "Humans": {
274             "SO": 60000,
275             "IO": 20,
276             "beta": beta,
277             "gamma": gamma,
278             "mu": mu,
279             "infectiousness": 1.0,
280             "active": True
281         },
282
283         "Orcs": {
284             "SO": 25000,
285             "IO": 0,
286             "beta": beta * 1.4,       # higher infection rate
287             "gamma": gamma,
288             "mu": mu * 0.5,          # lower death rate
289             "infectiousness": 1.2,
290             "active": True
291         },
```

```
        "Elves": {
            "S0": 15000,
            "I0": 0,
            "beta": beta,
            "gamma": gamma * 1.7,   # higher recovery rate
            "mu": mu,
            "infectiousness": 1.0,
            "active": True
        },

        "NPCs": {
            "S0": 0,
            "I0": 100,
            "beta": 0.0,
            "gamma": 0.0,
            "mu": 0.0,
            "infectiousness": 1.5,  # they infect others strongly
            "active": False         # cannot recover, die, or be infected
        }
    }

    total_population = sum(groups[g]["S0"] + groups[g]["I0"] for g in groups)

    # times, states, events, group_names = simulate_grouped_sisd(
    #     groups=groups,
    #     t_max=160
    # )

    # plot_grouped_sisd(times, states, group_names)
    # plot_group_infections(times, states, group_names)
    # plot_group_deaths(times, states, group_names)

    n_simulations = 1

    results = run_many_grouped_simulations(
        groups=groups,
        n_simulations=n_simulations,
        t_max=160
    )

    print_grouped_summary(results, total_population)


if __name__ == "__main__":
    main()
```

.8 Task 2c

```
def simulate_network_sisd(N_list, I0_list, travel_rate, beta, gamma, mu, t_max,
                          travel_factor_I=1.0, rng=None):
    """
    SISD dynamics on a network of K cities connected by travel routes.

    Parameters
    ----------
    N_list, I0_list : length-K lists, starting population / starting infected per city.
```

```
    travel_rate      : K x K matrix. travel_rate[a, b] is the per-capita rate at which a
                       SUSCEPTIBLE player in city a moves to city b (0 means "no direct
                       route from a to b"). Read it row-by-row: row a = "routes leaving
                       ↪ a".
    beta, gamma, mu  : the usual infection / recovery / death rates, shared by every
    ↪ city.
    travel_factor_I  : infected players travel at travel_rate * travel_factor_I instead
    ↪ of
                       travel_rate. <1 = sick players mostly stay put; >1 = panicked
                       ↪ fleeing.
    t_max            : stop the simulation at this time even if the epidemic is still
    ↪ going
                       (network runs can persist a while as cities re-seed each other).
    rng              : numpy random Generator, for reproducibility.

    Returns
    -------
    times  : array of event times.
    states : array with one row per event time and 3*K columns, laid out as
             [S_0, ..., S_{K-1}, I_0, ..., I_{K-1}, D_0, ..., D_{K-1}].
    K      : number of cities (handed back just for convenience when slicing `states`).
    """
    rng = np.random.default_rng() if rng is None else rng
    K = len(N_list)

    # One S/I/D triple per city, all as plain 1D numpy arrays of length K so that every
    # "local" rate below can be computed for all K cities at once (no per-city
    ↪ for-loop).
    S = np.array([N_list[k] - I0_list[k] for k in range(K)], dtype=float)
    I = np.array(I0_list, dtype=float)
    D = np.zeros(K)
    travel_rate = np.array(travel_rate, dtype=float)

    # Pre-compute the list of "real" routes ONCE, outside the simulation loop: every (a,
    ↪ b) pair
    # with a nonzero travel rate. We do this up front (rather than re-scanning the whole
    ↪ K x K
    # matrix on every single event) purely for speed | with K=5 cities this list might
    ↪ only have
    # a handful of entries even though the matrix has 25 cells, most of them zero.
    edges = [(a, b) for a in range(K) for b in range(K) if a != b and travel_rate[a, b]
    ↪ > 0]
    n_edges = len(edges)

    t = 0.0
    # states[i] is a length-3K snapshot [S_0..S_{K-1}, I_0..I_{K-1}, D_0..D_{K-1}] right
    ↪ after
    # the i-th event (or at t=0 for the very first row). Concatenating into one flat
    ↪ vector
    # (rather than keeping three separate K-length arrays) makes it trivial to dump
    ↪ everything
    # into a single 2D numpy array at the end for plotting/slicing.
    times, states = [t], [np.concatenate([S, I, D]).copy()]

    while t < t_max and I.sum() > 0:
        # ---- Step 1: compute every possible event's rate, all at once, as plain arrays
        ↪ ----
        N_alive = S + I  # dead players aren't part of anyone's contact network (as in
        ↪ Part 0)

```

```
        # Local SISD events, one rate per city (length-K arrays):
        infection_rate = beta * S * I / np.maximum(N_alive, 1e-12)   # S_k -> I_k
        recovery_rate = gamma * I                                    # I_k -> S_k
        death_rate = mu * I                                          # I_k -> D_k

        # Travel events, one rate per directed edge (length-n_edges arrays). travel_S[e] is the
        # rate of "a susceptible leaves edges[e][0] and arrives at edges[e][1]"; travel_I[e] is
        # the same for an infected player, scaled by travel_factor_I.
        travel_S = np.array([travel_rate[a, b] * S[a] for a, b in edges])
        travel_I = np.array([travel_rate[a, b] * travel_factor_I * I[a] for a, b in edges])

        # ---- Step 2: stack every rate into one flat array, in a FIXED, KNOWN order ----
        # [ K infection rates | K recovery rates | K death rates | n_edges travel_S | n_edges travel_I ]
        # Remembering this exact layout is what lets us decode `idx` back into "which event,
        # which city/edge" further down | the order here must match the `if/elif` chain below.
        rates = np.concatenate([infection_rate, recovery_rate, death_rate, travel_S, travel_I])
        total_rate = rates.sum()
        if total_rate <= 0:
            break  # nothing left that can possibly happen (e.g. I hit 0 everywhere)

        # ---- Step 3: WHEN does the next event happen? ----
        # One exponential draw with the TOTAL rate, exactly as in every earlier section -
        # this is rule 1 from the markdown above, just with "total_rate" now summing over
        # every city and every travel route instead of just 3 numbers.
        dt = rng.exponential(1.0 / total_rate)
        if t + dt > t_max:
            # Censor the run: hold the current state at exactly t_max instead of overshooting
            # (same fix as Part 0's t_max handling).
            times.append(t_max)
            states.append(np.concatenate([S, I, D]).copy())
            break
        t += dt

        # ---- Step 4: WHICH event happens? (the generic version of rule 2 above) ----
        cum = np.cumsum(rates)            # running totals: cum[j] = r_0 + r_1 + ... + r_j
        u = rng.random() * total_rate     # uniform draw on [0, total_rate)
        idx = int(np.searchsorted(cum, u))  # index of the first rate-bucket that "contains" u

        # Decode idx back into an actual event using the SAME segment order as the
        # concatenation above: [infection x K | recovery x K | death x K | travel_S x edges | travel_I x edges]
        if idx < K:
            # idx in [0, K): infection in city idx
            k = idx
            S[k] -= 1; I[k] += 1
        elif idx < 2 * K:
            # idx in [K, 2K): recovery in city (idx - K)
```

```
            k = idx - K
            I[k] -= 1; S[k] += 1
        elif idx < 3 * K:
            # idx in [2K, 3K): death in city (idx - 2K)
            k = idx - 2 * K
            I[k] -= 1; D[k] += 1
        elif idx < 3 * K + n_edges:
            # idx in [3K, 3K + n_edges): a SUSCEPTIBLE travels along edge number (idx -
            ↪  3K)
            a, b = edges[idx - 3 * K]
            S[a] -= 1; S[b] += 1
        else:
            # idx in [3K + n_edges, 3K + 2*n_edges): an INFECTED travels along
            # edge number (idx - 3K - n_edges)
            a, b = edges[idx - 3 * K - n_edges]
            I[a] -= 1; I[b] += 1

        times.append(t)
        states.append(np.concatenate([S, I, D]).copy())

    return np.array(times), np.array(states, dtype=float), K




cities = ["Orgrimmar", "Ironforge", "Stormwind", "Undercity", "Thunder Bluff"]
K = len(cities)
city_N = [600, 400, 400, 300, 300]   # starting population of each city
city_I0 = [20, 0, 0, 0, 0]           # the raid group hearths back to Orgrimmar:
↪  outbreak starts there only

# travel[a, b] = rate at which a susceptible player in city a heads to city b (per day).
# Row 0 (Orgrimmar) is nonzero in every column: it's the hub. Rows 1-4 only connect to
↪  Orgrimmar
# plus one neighbour each, e.g. Ironforge <-> Stormwind (both Alliance capitals, on the
↪  same
# continent) and Undercity <-> Thunder Bluff (both Horde, on the same continent).
travel = np.array([
#    Org   IF    SW    UC    TB
    [0,    0.02, 0.02, 0.01, 0.01],   # from Orgrimmar
    [0.02, 0,    0.01, 0.0,  0.0 ],   # from Ironforge
    [0.02, 0.01, 0,    0.0,  0.0 ],   # from Stormwind
    [0.01, 0.0,  0.0,  0,    0.01],   # from Undercity
    [0.01, 0.0,  0.0,  0.01, 0   ],   # from Thunder Bluff
])

rng = np.random.default_rng(70)
t_net, st_net, K = simulate_network_sisd(city_N, city_I0, travel, BETA, GAMMA, MU,
                                         t_max=120, travel_factor_I=1.0, rng=rng)

# st_net has 3*K=15 columns laid out as [S_0..S_4, I_0..I_4, D_0..D_4] (see the
↪  function's
# docstring) | so "infected count in city k" is column K+k, not column k.
fig, ax = plt.subplots(figsize=(9, 5))
for k in range(K):
    ax.plot(t_net, st_net[:, K + k], label=cities[k])
ax.set_xlabel("Time (days)"); ax.set_ylabel("Infected players")
ax.set_title("Corrupted Blood spreading from Orgrimmar to other capitals via portals")
```

```
ax.legend(fontsize=8)
savefig(fig, "part2c_network_timeseries.png")
plt.show()
```

.9 Task 2d

```

rng = np.random.default_rng(42)

def gillespie_vacc(N, I0, beta, gamma, mu, nu, eff, t_max=60, max_events=400000):
    S, V, I, D = N - I0, 0, I0, 0
    t, ev, peak = 0.0, 0, I
    while I > 0 and t < t_max and ev < max_events:
        N_live = S + V + I
        infS = beta * S * I / N_live if N_live > 0 else 0.0
        infV = beta * (1-eff) * V * I / N_live if N_live > 0 else 0.0
        vac  = nu * S
        rec  = gamma * I
        dth  = mu * I
        Rt = infS + infV + vac + rec + dth
        if Rt == 0:
            break
        t += rng.exponential(1 / Rt)
        u = rng.random() * Rt
        if   u < infS:               S -= 1; I += 1
        elif u < infS+infV:          V -= 1; I += 1
        elif u < infS+infV+vac:      S -= 1; V += 1
        elif u < infS+infV+vac+rec:  I -= 1; S += 1
        else:                        I -= 1; D += 1
        peak = max(peak, I); ev += 1
    return peak, D

def outbreak_size(N, I0, beta, gamma, mu, nu, eff, n_runs=400):
    res = [gillespie_vacc(N, I0, beta, gamma, mu, nu, eff) for _ in range(n_runs)]
    peaks  = [r[0] for r in res]
    deaths = [r[1] for r in res]
    return np.mean(peaks), np.mean(deaths)

N, I0, beta, gamma, mu = 1000, 20, 0.8, 0.3, 0.15

nu = 0.6
effs = [0.0, 0.2, 0.4, 0.6, 0.8, 1.0]
pk_e, d_e = zip(*[outbreak_size(N, I0, beta, gamma, mu, nu, e) for e in effs])


eff = 0.9
nus = [0.0, 0.01, 0.05, 0.08, 0.1, 0.15, 0.2, 0.3, 0.5]
pk_n, d_n = zip(*[outbreak_size(N, I0, beta, gamma, mu, n_, eff) for n_ in nus])

fig, ax = plt.subplots(1, 2, figsize=(12, 4))
ax[0].plot(effs, pk_e, 'o-', label='peak infected')
ax[0].plot(effs, d_e, 's-', label='total deaths')
ax[0].set_xlabel("vaccine effectiveness"); ax[0].set_ylabel("count")
ax[0].set_title("Effectiveness vs outbreak (nu=0.3)"); ax[0].legend()
ax[1].plot(nus, pk_n, 'o-', label='peak infected')
ax[1].plot(nus, d_n, 's-', label='total deaths')
ax[1].set_xlabel("vaccination rate nu"); ax[1].set_ylabel("count")
```

```
ax[1].set_title("Vaccination speed vs outbreak (eff=0.9)"); ax[1].legend()
plt.tight_layout(); plt.show()

import numpy as np
rng = np.random.default_rng(7)

def run(N, I0, beta, gamma, mu, nu, eff, rng, t_max=200, max_events=400000):
    S, V, I, D = N - I0, 0, I0, 0
    t, ev = 0.0, 0
    while I > 0 and t < t_max and ev < max_events:
        Nl = S+V+I
        infS = beta*S*I/Nl if Nl>0 else 0.0
        infV = beta*(1-eff)*V*I/Nl if Nl>0 else 0.0
        vac, rec, dth = nu*S, gamma*I, mu*I
        Rt = infS+infV+vac+rec+dth
        if Rt == 0: break
        t += rng.exponential(1/Rt)
        u = rng.random()*Rt
        if   u < infS:              S-=1; I+=1
        elif u < infS+infV:         V-=1; I+=1
        elif u < infS+infV+vac:     S-=1; V+=1
        elif u < infS+infV+vac+rec: I-=1; S+=1
        else:                       I-=1; D+=1
    return D

N, I0, beta, gamma, mu, nu, n = 1000, 20, 0.8, 0.3, 0.15, 0.6, 500

print(f"{'eff':>5}{'mean':>9}{'std':>9}{'95% CI':>12}")
for eff in [0.0, 0.2, 0.4, 0.6, 0.8]:
    D = np.array([run(N, I0, beta, gamma, mu, nu, eff, rng) for _ in range(n)])
    mean = D.mean()
    std  = D.std(ddof=1)
    half = 1.96 * std / np.sqrt(n)
    print(f"{eff:>5.1f}{mean:>9.1f}{std:>9.1f}  ±{half:>8.2f}")

```